

Guide complet des paramètres d'entraînement

Réseau de neurones multi-sorties pour la classification des recommandations

Ce document explique chaque paramètre utilisé pour construire, entraîner et évaluer le modèle de recommandations (63 recommandations, 4 niveaux de priorité), et donne des repères concrets pour les régler afin d'améliorer le modèle.

1. Forme des données : X et Y

1.1 Les features (X)

X représente les réponses au questionnaire. Chaque exemple est un profil de 5 réponses (une par question), chaque réponse valant 1, 2, 3 ou 4.

Objet	Shape	Signification
Un profil	(5,)	5 réponses (Q1 à Q5) — un vecteur
X complet	(n_exemples, 5)	Tous les profils empilés — une matrice

1.2 La sortie (Y)

Y représente la priorité prédite pour chacune des 63 recommandations. Chaque position dans le vecteur correspond à une recommandation précise (via LISTE_RECOS), et la valeur stockée est le code de sa classe de priorité (0=Absente, 1=Moyenne, 2=Elevée, 3=Critique).

Objet	Shape	Signification
cible / vraie_cible	(63,)	Priorités des 63 recos pour UN profil — vecteur
Y complet	(n_exemples, 63)	Toutes les cibles empilées — matrice
Y[:, i]	(n_exemples,)	Priorité de la recommandation i, pour tous les profils

Pourquoi cette forme est importante : le réseau a une entrée de taille fixe shape=(5,) (keras.Input) et 63 sorties softmax séparées (une par recommandation). Keras attend donc Y sous forme d'une liste de 63 tableaux (Y_par_tete = [Y[:, i] for i in range(NB_RECOS)]), un tableau par tête de sortie.

2. Paramètres de fit() : epochs, batch_size, validation_split

2.1 epochs

Un epoch = un passage complet du modèle sur l'ensemble des données d'entraînement. Plus il y a d'epochs, plus le modèle a d'occasions d'ajuster ses poids.

Cas	Effet	Conséquence
Trop petit (ex: 5)	Le modèle n'a pas le temps d'apprendre	Sous-apprentissage (underfitting) — accuracy faible partout
Trop grand (ex: 500)	Le modèle mémorise les données d'entraînement	Sur-apprentissage (overfitting) — bon sur les données connues, mauvais ailleurs
Bien réglé	val_loss baisse puis se stabilise	Bon compromis apprentissage / généralisation

2.2 batch_size

Nombre d'exemples traités avant chaque mise à jour des poids du réseau.

Cas	Effet	Conséquence
Petit (8-16)	Mises à jour fréquentes mais plus bruitées	Entraînement plus lent, mais parfois meilleure généralisation

Grand (256-512)	Mises à jour stables et rapides	Utilise mieux le CPU/GPU, mais risque de moins bien généraliser
Compromis recommandé	32 à 64 pour ~1800 exemples	Stable et efficace

2.3 validation_split

Fraction des données mise de côté pour évaluer le modèle sur des exemples qu'il n'a pas utilisés pour apprendre. Sert à détecter le surapprentissage.

Cas	Effet	Conséquence
Petit (0.05)	Plus de données pour apprendre	Évaluation moins fiable (peu d'exemples de test)
Grand (0.3)	Évaluation plus robuste	Moins de données pour apprendre — modèle potentiellement moins performant
Recommandé (~1800 ex.)	0.15 à 0.2	270 à 360 exemples de validation — bon compromis

3. Le paramètre verbose

Contrôle uniquement l'affichage dans la console pendant fit()/predict() — n'a aucun effet sur les performances du modèle.

Valeur	Nom	Affichage
0	Silencieux	Rien ne s'affiche
1	Barre de progression	Barre animée + métriques à chaque batch
2	Une ligne par epoch	Résumé texte, sans animation — utile en notebook/logs

Conseil : utiliser verbose=2 pendant le développement pour suivre loss/val_loss à chaque epoch, et verbose=0 pour un script final/automatisé.

4. EarlyStopping

Callback qui surveille l'entraînement à chaque epoch et peut l'arrêter automatiquement pour éviter le surapprentissage, au lieu de fixer un nombre d'epochs à l'aveugle.

```
early_stop = EarlyStopping(monitor="val_loss", patience=15, restore_best_weights=True)
hist = modele.fit(X, Y_par_tete, epochs=150, batch_size=32,
                 validation_split=0.15, callbacks=[early_stop], verbose=2)
```

Paramètre	Rôle	Importance
monitor="val_loss"	Surveille la perte sur les données de validation	Évite de se fier à une bonne performance trompeuse sur le training
patience=15	Attend 15 epochs sans amélioration avant d'arrêter	Évite l'arrêt prématuré à cause d'une fluctuation normale
restore_best_weights=True	Restaure les poids du meilleur epoch (passé)	Essentiel pour garantir de garder la meilleure version du modèle

Point de vigilance : le callback doit obligatoirement être passé dans callbacks=[early_stop] lors de l'appel à fit(). S'il est seulement créé mais jamais passé à fit(), il n'a strictement aucun effet et l'entraînement se déroule normalement sans arrêt anticipé.

5. Évaluer le modèle : accuracy et matrice de confusion

5.1 Deux accuracy différentes, et pourquoi

Sur les 63 recommandations, la plupart sont "Absente" pour un profil donné. Un modèle qui prédirait "Absente" partout, sans rien apprendre d'utile, obtiendrait déjà une accuracy globale trompeusement élevée. D'où l'intérêt de calculer deux métriques :

Métrique	Ce qu'elle mesure	Piège
Accuracy globale	Sur les 63 sorties, Absente incluse	Trompeuse si beaucoup de Absente (facile à prédire)
Accuracy sur recos actives	Uniquement sur les recommandations réellement incluses du modèle	Reflecte mieux la vraie qualité

5.2 Matrice de confusion

Tableau croisant la vraie classe (lignes) et la classe prédite (colonnes) pour les 4 priorités (Absente, Moyenne, Elevee, Critique). La diagonale montre les prédictions correctes ; les autres cases montrent les erreurs (ex : le modèle prédit "Critique" alors que la vraie priorité était "Elevee").

Elle permet de voir si le modèle confond systématiquement deux priorités proches (par exemple Elevee et Critique), ce qu'une simple accuracy globale ne montre pas.

6. Récapitulatif : valeurs recommandées (~1800 exemples)

Paramètre	Valeur recommandée
batch_size	32
validation_split	0.15 à 0.2
epochs	100-150, avec EarlyStopping(patience=15)
verbose	2 en développement, 0 en script final
Architecture	Ajouter du Dropout (0.2-0.3) si écart train/validation important

En résumé : les paramètres d'entraînement (epochs, batch_size, validation_split) règlent l'équilibre entre apprentissage et généralisation ; EarlyStopping automatise ce réglage pour epochs ; verbose ne change que l'affichage ; et l'accuracy sur les recommandations actives + la matrice de confusion donnent l'évaluation la plus fidèle de la vraie performance du modèle.